Universidade Catolica de Brasilia
Engenharia de Software

Lucas Naunn Castro Passos
Ludmilla Oliveira Gama
Miguel Dourado de Jesus
Miguel Guimarães Galvão
Rick Guedes da Silva
Thiago de Sousa Benvenuto

Laboratório de banco de dados

Achados e perdidos

Brasília - DF
2026

Sistema de Achados e Perdidos Universitário

Autores: Lucas Naunn Castro Passos; Ludmilla Oliveira Gama; Miguel Dourado de Jesus; Miguel Guimarães Galvão; Rick Guedes da Silva e Thiago de Sousa Benvenuto.

Instituição: Universidade Católica de Brasília

Disciplina: Laboratório de Banco de Dados

Professor: Jefferson

Este artigo apresenta o Sistema de Achados e Perdidos, uma aplicação desktop desenvolvida em Java para registrar, consultar, reivindicar e devolver objetos encontrados no ambiente universitário e suas dependencias. O foco do trabalho é o banco de dados achados_perdidos, modelado em MySQL com 12 tabelas, dois índices de busca, duas views, duas triggers, duas procedures, duas functions e quatro perfis de acesso com privilégios distintos. São descritos o funcionamento do sistema, as tecnologias adotadas, o Diagrama Entidade-Relacionamento (DER) e a justificativa técnica de cada recurso do banco empregado.

Palavras-chave: achados e perdidos; MySQL; modelagem de dados; triggers; Java.

1. Introdução

Em campi universitários, objetos como cadernos, carregadores, documentos e garrafas são esquecidos diariamente. Quando não há um processo organizado, a devolução depende da memória de quem recebeu o objeto, não deixa registro e abre espaço para entregas indevidas: sem critério formal, nada garante que quem retira o item é de fato o dono.

O Sistema de Achados e Perdidos foi construído para formalizar esse fluxo. Um funcionário cadastra o objeto encontrado; o aluno consulta os objetos disponíveis e abre uma reivindicação; o funcionário aprova ou recusa o pedido e, por fim, registra a devolução com assinatura do aluno. Cada etapa deixa rastro no banco de dados, e relatórios por período permitem acompanhar o volume de objetos e de pedidos.

O banco de dados é o centro da solução, pois é nele que ficam as regras que não podem depender apenas da interface: unicidade de CPF e e-mail, impossibilidade de uma reivindicação nascer já aprovada, atualização automática do status do objeto após a devolução e limitação do que cada perfil pode ler ou alterar. Por isso, este artigo dá ênfase à modelagem e à justificativa dos recursos de banco utilizados.

O texto está organizado assim:

Seção 2 - Apresenta os objetivos.

Seção 3 - Metodologia.

Seção 4 - Sistema.

Seção 5 - Modelagem do banco de dados.

Seção 6 - Conclusão.

2. Objetivos

Objetivo geral

Desenvolver um sistema de informação para gerenciar o ciclo completo de objetos achados e perdidos em uma universidade, apoiado em um banco de dados relacional íntegro, seguro e consultável.

Objetivos específicos

1. Modelar o domínio por meio de um DER e traduzi-lo em tabelas MySQL com chaves, restrições e tipos adequados.
2. Registrar o ciclo objeto > reivindicação > devolução, com protocolos legíveis e histórico de ações.
3. Diferenciar três perfis de usuário: aluno, funcionário e administrador. Tanto na aplicação quanto nos privilégios do banco.
4. Acelerar as buscas de objetos por categoria e período por meio de índices.
5. Mover para o servidor as regras de consistência e os cálculos de relatório, usando triggers, procedures e functions.
6. Oferecer relatórios por período sobre objetos cadastrados e reivindicações.

3. Metodologia

O trabalho foi conduzido em etapas, e cada etapa deixou um artefato verificável no projeto:

1. Levantamento do fluxo e dos perfis. Foram identificados três perfis: aluno, funcionário, administrador e o cadastro > consulta > reivindicação > aprovação > devolução > relatório.
2. Modelagem em três níveis. Foram elaborados o modelo conceitual (DER), o modelo lógico (relacional) e o modelo físico em MySQL.
3. Implementação do banco por scripts versionados, numerados na ordem de execução, na pasta database: 01_banco.sql (tabelas); 02_inserts.sql (dados de exemplo); 03_grupos_usuarios.sql (grupos e vínculo dos usuários); 04_indices.sql; 05_views.sql; 06_procedures.sql; 07_triggers.sql; e 08_seguranca_e_identificadores.sql (usuários do banco, privilégios e functions). Os dados de exemplo entram antes das triggers para que a regra de reivindicação pendente não altere o histórico, e os privilégios são concedidos por último, quando views e procedures já existem.
4. Desenvolvimento da aplicação em Java 21 com NetBeans, organizada em três pacotes: br.com.visao, br.com.controller e br.com.entidade.
5. Validação. Os dados de exemplo cobrem todos os perfis inclusive administrador que também é funcionário. O script de índices inclui um EXPLAIN para conferir o plano de execução da busca filtrada, e o projeto traz estrutura de testes JUnit 5 para a classe PeriodoRelatorio.
6. Controle de versão com Git, em repositório compartilhado pelo grupo.

4. Descrição do Sistema

Funcionalidades

O sistema é uma aplicação desktop de 14 telas. Abaixo, as funcionalidades agrupadas por perfil, conforme implementado no código.

- Login por e-mail institucional e senha. O controller só aceita domínios @a.ucb.br e @p.ucb.br. Após autenticar, a classe SessaoUsuario guarda ID, nome, CPF e os indicadores de funcionário e administrador, e a aplicação abre a tela correspondente ao perfil.
- Autocadastro de usuário, com validação de domínio de e-mail e de CPF.

Aluno

- Procurar objetos, com filtros por nome, categoria e intervalo de datas.
- Reivindicar o objeto. O sistema recusa a reivindicação se o objeto já foi devolvido ou encaminhado para doação, e também se o mesmo retirante já reivindicou aquele objeto. Se o retirante ainda não estiver na base, ele é cadastrado a partir do nome e do documento. Ao final, é gerado um protocolo no formato REIV-AAAA-NNNNNN.
- Minhas reivindicações, com pesquisa por objeto e contadores de solicitadas, aceitas, recusadas e objetos devolvidos.
- Painel inicial com objetos recentes e totais de objetos encontrados, requisitados e devolvidos.

Funcionário

- Cadastrar objeto, com descrição genérica, detalhes sensíveis, categoria, data e hora do achado, campus, bloco, ambiente e data limite de guarda. Data e hora de cadastro são preenchidas pelo servidor **(CURDATE()** e **CURTIME())**, e o usuário responsável é identificado pelo e-mail informado.
- Consultar objetos, com os mesmos filtros do aluno, além de editar, excluir e alterar o status entre Disponível, Em análise de posse e Encaminhado para doação.
- Gerenciar reivindicações: listar, pesquisar, aprovar e recusar. A mudança de status só é aceita quando a reivindicação ainda está pendente.
- Registrar devolução, vinculada a uma reivindicação, com a assinatura digital do aluno, geração do protocolo DEV-AAAA-NNNNNN e registro na auditoria.
- Gerar relatórios por período: objetos cadastrados e reivindicações.

Administrador

- Gerenciar funcionários, pesquisar usuários, promover um usuário a funcionário, atualizar cargo e setor, remover a condição de funcionário e excluir usuários comuns. A exclusão só atinge quem não é funcionário nem administrador, regra aplicada na própria instrução **DELETE**.

A tabela **Log_Auditoria** registra quem fez cada reivindicação e cada devolução, com data e hora do servidor.

4.2 Tecnologias utilizadas e justificativas

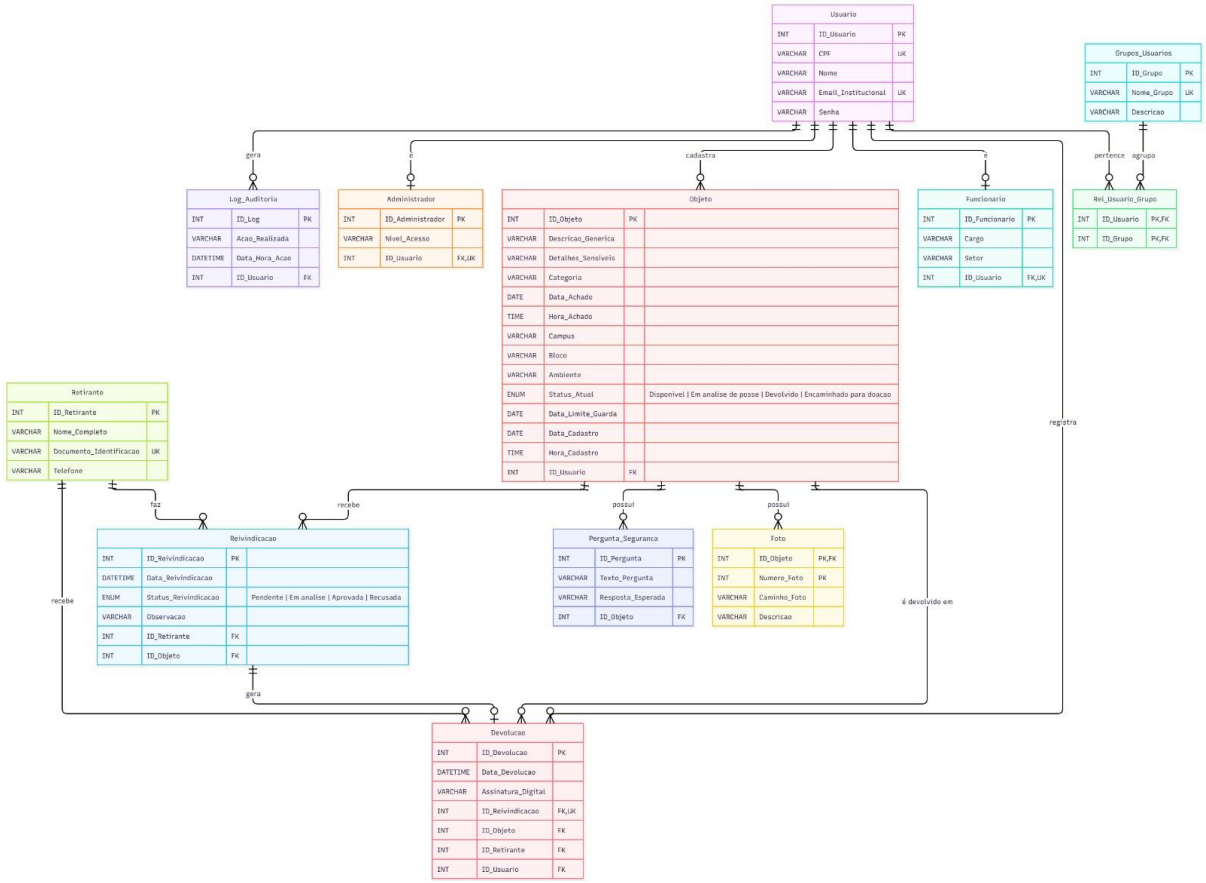
O sistema não possui camada web, o frontend é a interface Swing, e o backend é formado pelas camadas Java de controle e acesso a dados, em conjunto com o MySQL.

Tecnologia	**Papel no projeto**	**Justificativa**
Java 21	Linguagem de toda a aplicação (javac.source e javac.target = 21)	Orientação a objetos permite separar telas, regras e acesso a dados em classes. A API java.time (LocalDate, LocalTime) mapeia diretamente para DATE e TIME do MySQL, o que reduz erros de conversão nas datas de achado, guarda e devolução.
Java Swing + GUI Builder do NetBeans (AbsoluteLayout)	14 telas em br.com.visao (arquivos .java e .form)	A aplicação desktop dispensa servidor web e navegador, o que combina com o uso em um balcão de atendimento. O GUI Builder agilizou a montagem das telas e deixou o projeto editável pelo grupo.
JDBC + MySQL Connector/J 8.0.33	Classe DAO (driver com.mysql.cj.jdbc.Driver, URL jdbc:mysql://localhost:3306/achados_perdidos)	É o padrão do Java para bancos relacionais. PreparedStatement com parâmetros evita SQL injection, e CallableStatement permite chamar as procedures de relatório.
MySQL 8 com InnoDB	Armazenamento e regras de integridade	Todas as tabelas usam ENGINE=InnoDB, que oferece chaves estrangeiras, transações e travamento por linha. O MySQL também oferece triggers, views, procedures, functions, ENUM e controle de usuários e privilégios, recursos exigidos pelo projeto.

Arquitetura em camadas (visão, controller, entidade)	Pacotes br.com.visao, br.com.controller, br.com.entidade	Separa a interface das regras e do SQL: as telas chamam controllers, e só as classes de entidade falam com o banco. Mudanças em uma camada não obrigam reescrever as outras.
Apache NetBeans + Ant	Ambiente e build (build.xml, nbproject)	Ambiente integrado ao GUI Builder e ao JUnit, usado por todos os integrantes.
JUnit 5	Pasta test	Permite testar regras de negócio isoladas do banco, como a classe PeriodoRelatorio.

5. Modelagem do Banco de Dados

Diagrama Entidade-Relacionamento (DER)



Entidades

O banco tem 12 tabelas. Os nomes abaixo seguem os scripts SQL.

- **Usuário:** pessoa cadastrada no sistema. PK ID_Usuario; CPF e Email_Institucional são UNIQUE; Senha é VARCHAR(255). É a entidade-base de aluno, funcionário e administrador.
- **Funcionário:** especialização de Usuário, com Cargo e Setor. PK própria ID_Funcionario; FK ID_Usuario com NOT NULL UNIQUE.
- Administrador: especialização de Usuário, com Nivel_Acesso. Mesma estrutura de FK única.
- **Objeto:** item achado. Guarda a Descricao_Generica (informação pública) separada de Detalhes_Sensiveis (destinada à conferência da posse), além de categoria, data e hora do achado, local (Campus, Bloco, Ambiente), Data_Limite_Guarda, data e hora de cadastro e Status_Atual (ENUM: Disponivel, Em analise de posse, Devolvido, Encaminhado para doacao; padrão *Disponivel*). A FK ID_Usuario indica quem cadastrou.
- **Foto:** imagens do objeto. É entidade fraca: sua PK é composta por ID_Objeto, Numero_Foto, pois uma foto só existe em função de um objeto.
- **Pergunta_Seguranca:** perguntas e respostas esperadas associadas a um objeto, destinadas a validar a posse por parte de quem o reivindica.
- Retirante: pessoa que reivindica ou retira um objeto Nome_Completo, Documento_Identificacao único e Telefone. Fica separado de usuário porque representa o papel de quem retira, identificado por documento.
- **Reivindicacao:** pedido de um retirante por um objeto, com Data_Reivindicacao DATETIME, Status_Reivindicacao ENUM: Pendente, Em analise, Aprovada, Recusada; padrão Pendente e Observação.
- **Devolução:** registro formal da entrega, com Data_Devolucao e Assinatura_Digital. Referência o objeto, o retirante, o funcionário que registrou ID_Usuario e a reivindicação de origem ID_Reivindicacao, UNIQUE.
- **Log_Auditoria:** histórico de ações Acao_Realizada, Data_Hora_Acao vinculado ao usuário que as realizou.
- **grupos_usuarios:** catálogo de grupos Administrador, Funcionario, Aluno, com nome_grupo único.
- **rel_usuario_grupo:** tabela associativa entre Usuario e grupos_usuarios, com PK composta por id_usuario, id_grupo.

Relacionamentos e cardinalidades

Relacionamento	Cardinalidade	Implementação
Usuario – Funcionario (especialização)	1 : 0..1	Funcionario.ID_Usuario NOT NULL UNIQUE
Usuario – Administrador (especialização)	1 : 0..1	Administrador.ID_Usuario NOT NULL UNIQUE
Usuario cadastra Objeto	1 : N	Objeto.ID_Usuario NOT NULL
Objeto possui Foto	1 : N (Foto é entidade fraca)	PK composta (ID_Objeto, Numero_Foto)
Objeto tem Pergunta_Seguranca	1 : N	Pergunta_Seguranca.ID_Objeto
Retirante faz Reivindicacao	1 : N	Reivindicacao.ID_Retirante
Objeto é alvo de Reivindicacao	1 : N	Reivindicacao.ID_Objeto
Reivindicacao resulta em Devolucao	1 : 0..1	Devolucao.ID_Reivindicacao UNIQUE
Devolucao – Objeto / Retirante	N : 1 cada	Devolucao.ID_Objeto, Devolucao.ID_Retirante
Usuario registra Devolucao	1 : N	Devolucao.ID_Usuario
Usuario gera Log_Auditoria	1 : N	Log_Auditoria.ID_Usuario
Usuario – grupos_usuarios	N : M	tabela associativa rel_usuario_grupo

Índices

O script de índices cria dois índices na tabela Objeto:

- idx_objeto_categoria_data sobre (Categoria, Data_Achado);
- idx_objeto_data_achado sobre (Data_Achado).

O método buscarObjetos monta uma consulta com filtro opcional de categoria por igualdade, intervalo de Data_Achado e ordenação ORDER BY Data_Achado DESC. O índice composto atende exatamente esse padrão: a igualdade na primeira coluna e a faixa na segunda permitem ler só o trecho do índice relevante, já em ordem de data. Pela regra do prefixo mais à esquerda, o mesmo índice também serve para filtrar só por categoria. Como esse prefixo não ajuda quando o filtro é só por data, foi criado o segundo índice, de coluna única. O script traz um EXPLAIN da consulta categoria para conferir o plano de execução.

O banco também conta com índices criados implicitamente pelas restrições: PKs, UNIQUE de CPF, Email_Institucional, Documento_Identificacao e ID_Reivindicacao, e os índices das FKs que o InnoDB cria automaticamente. Eles sustentam operações frequentes do código, como o login por e-mail LoginEntity e a busca de retirante por documento RetiranteEntity.buscarPorDocumento.

Não foi criado índice em Descricao_Generica: o filtro por nome usa LIKE '%texto%', com curinga no início, padrão que um índice B-tree não consegue aproveitar. Também não se indexou Status_Atual, que possui apenas quatro valores e, portanto, baixa seletividade.

Views

- reivindicacoes_por_retirante: junta Reivindicacao e Objeto e expõe ID, data e status da reivindicação, além de descrição, categoria e campus do objeto. É usada para listar as reivindicações de um retirante (listarPorRetirante).
- consultar_reivindicacoes: junta Reivindicacao, Retirante e Objeto e expõe os dados completos para a tela de gerenciamento. Também é usada na busca de uma reivindicação por ID, durante o registro da devolução.

Reuso e simplicidade: o JOIN é escrito uma vez no banco, e o Java consulta uma estrutura que parece uma tabela, sem repetir INNER JOIN em vários métodos; os nomes das views seguem os dos métodos da aplicação para facilitar a manutenção. (2) Segurança por projeção: a primeira view deixa de fora Detalhes_Sensiveis e os dados pessoais de outros retirantes, e os privilégios do script de segurança dão ao aluno SELECT apenas nela, e ao funcionário SELECT na segunda. (3) Evolução: se a estrutura das tabelas mudar, ajusta-se a view e o código Java permanece.

Triggers

- trg_reivindicacao_pendente (BEFORE INSERT em Reivindicacao): força Status_Reivindicacao = 'Pendente', qualquer que seja o valor enviado.

- trg_objeto_devolvido (AFTER INSERT em Devolucao): executa UPDATE Objeto SET Status_Atual = 'Devolvido' para o objeto da devolução.

A primeira trigger vai além do DEFAULT 'Pendente' da coluna, que só vale quando o campo é omitido. O perfil aluno possui INSERT em Reivindicacao; sem a trigger, quem conseguisse executar SQL com esse usuário poderia criar um pedido já *Aprovada*. A regra "todo pedido nasce pendente" fica, assim, garantida pelo banco, e não pela boa-fé da aplicação. A segunda trigger garante que registrar a devolução e marcar o objeto como devolvido sejam uma operação só: o UPDATE ocorre na mesma transação do INSERT, de modo que não há devolução registrada com objeto ainda *Disponivel*. A regra continua válida mesmo que outra tela, script ou usuário insira devoluções, e o perfil funcionário não precisa de lógica extra no Java para isso. Também evita que o objeto volte a ser reivindicado, pois a aplicação recusa pedidos sobre objetos devolvidos.

Procedures e Functions

- sp_relatorio_objetos(p_data_inicial, p_data_final) devolve quatro conjuntos de resultados: total de objetos, total por categoria, total por campus e total por mês (ordem cronológica), filtrando por Data_Cadastro. É chamada pelo Java com {CALL sp_relatorio_objetos(?, ?)}.
- sp_relatorio_reivindicacoes(p_data_inicial, p_data_final) devolve, em uma linha, o total de reivindicações solicitadas, aprovadas, recusadas e pendentes, usando SUM(CASE ...).

Ambas validam os parâmetros: se apenas uma das datas for informada, disparam SIGNAL SQLSTATE '45000' com a mensagem *Informe as duas datas ou deixe ambas vazias*. Datas nulas significam "todo o período".

O relatório é composto por várias agregações sobre a mesma faixa de datas. Em procedure, o filtro é escrito uma vez, o cálculo acontece no servidor e a aplicação recebe só os totais, com uma única chamada em vez de várias consultas, o que diminui o tráfego entre Java e MySQL. A validação das datas fica no banco, mantendo a regra idêntica para qualquer cliente que chame o procedimento. O SIGNAL transforma o erro em exceção SQL, que o Java pode tratar e exibir ao usuário.

Functions

- fn_gerar_protocolo_devolucao(id) retorna textos como DEV-2026-000001.
- fn_gerar_protocolo_reivindicacao(id) retorna textos como REIV-2026-000045.

As duas combinam prefixo, ano da data do registro (YEAR(...)) e o ID preenchido com zeros à esquerda (LPAD(id, 6, '0')). São chamadas logo após o INSERT, por SELECT fn_gerar_protocolo).

O ID numérico é adequado para o banco, mas inadequado para o aluno guardar e informar. O protocolo é legível, identifica o tipo do documento e o ano e não exige coluna

nova, pois é derivado do ID e da data. Como *function*, ela retorna um valor e pode ser usada dentro de um SELECT; centralizar o formato no banco impede que duas telas gerem protocolos diferentes para o mesmo registro.

Geração de IDs e uso do AUTO_INCREMENT

As chaves primárias técnicas de Usuario, Funcionario, Administrador, Objeto, Pergunta_Seguranca, Retirante, Reivindicacao, Devolucao, Log_Auditoria e grupos_usuarios são geradas por AUTO_INCREMENT. Essas chaves não têm significado de negócio e nunca são exibidas nem digitadas pelo usuário; servem apenas para identificar a linha e ligar as tabelas por chave estrangeira. A escolha se justifica por três motivos:

- **Atomicidade e concorrência.** O InnoDB gera o próximo valor de forma atômica a cada INSERT, de modo que alunos, funcionários e administradores inserindo ao mesmo tempo, em conexões diferentes, nunca recebem o mesmo ID. Uma função própria baseada em MAX(id) + 1 teria condição de corrida: duas conexões poderiam calcular o mesmo valor antes de qualquer uma gravar, exigindo travamento extra da tabela.
- **Desempenho.** No InnoDB a tabela é organizada fisicamente pela chave primária (índice clusterizado). Um INT de 4 bytes sempre crescente faz cada nova linha entrar no fim da árvore B, sem reorganizar páginas, e mantém leves as chaves estrangeiras e os JOINs.
- **Simplicidade.** A aplicação recupera o ID gerado com getGeneratedKeys(), sem lógica adicional para descobrir o próximo valor livre.

Para os dados críticos do sistema, a reivindicação e a devolução, que comprovam a posse e a entrega do objeto, foi adotada uma regra própria de identificação, implementada pelas functions fn_gerar_protocolo_reivindicacao e fn_gerar_protocolo_devolucao descritas acima: prefixo do tipo de registro, ano e ID com seis dígitos (REIV-2026-000045, DEV-2026-000001). Esse protocolo é o identificador que o aluno recebe e apresenta no balcão. Por ser derivado do ID e da data, é único, determinístico e sempre igual para o mesmo registro, sem duplicar informação em uma coluna nova. Assim, o AUTO_INCREMENT fica restrito à identificação interna, e a identificação externa dos dados críticos segue a regra definida pelo grupo.

Duas tabelas não usam AUTO_INCREMENT, por decisão de modelagem: Foto, entidade fraca com chave composta (ID_Objeto, Numero_Foto), cuja numeração recomeça em cada objeto; e rel_usuario_grupo, cuja chave é formada pelas duas chaves estrangeiras (id_usuario, id_grupo). Já CPF, Email_Institucional e Documento_Identificacao são chaves candidatas, protegidas por UNIQUE contra duplicidade, mas não foram usadas como chave primária porque são dados pessoais (sujeitos à LGPD), podem precisar de correção e ocupam mais espaço que um INT, o que encareceria todas as chaves estrangeiras que os referenciam.

Segurança e controle de acesso

O script 08_seguranca_e_identificadores.sql cria quatro usuários de banco com privilégio mínimo:

Usuário	Privilégios principais
app_achados	SELECT, INSERT, UPDATE, DELETE no banco; usado antes do login e no cadastro de novo usuário, quando o perfil ainda é desconhecido
achados_admin	SELECT, INSERT, UPDATE, DELETE no banco
achados_funcionario	CRUD em Objeto; SELECT e UPDATE em Reivindicacao; SELECT e INSERT em Devolucao; SELECT em Usuario e Retirante; apenas INSERT em Log_Auditoria; SELECT na view consultar_reivindicacoes
achados_aluno	SELECT em Objeto; SELECT e INSERT em Retirante e Reivindicacao; SELECT em Devolucao; apenas INSERT em Log_Auditoria; SELECT na view reivindicacoes_por_retirante

Se a aplicação usasse sempre um único usuário com todos os poderes (como o root, que o comentário do próprio script indica ter sido substituído), qualquer falha de validação na interface se transformaria em acesso total aos dados. Com o princípio do menor privilégio, o aluno não consegue alterar objetos nem aprovar a própria reivindicação; o funcionário não exclui reivindicações nem devoluções; e os perfis comuns só inserem no log de auditoria, sem poder editá-lo ou apagá-lo, o que protege a confiabilidade do histórico. Os GRANT EXECUTE das functions também são concedidos por perfil.

Conclusão

O Sistema de Achados e Perdidos mostrou como um banco de dados bem modelado sustenta um processo que antes era informal. O DER organizou o domínio em entidades coerentes; as restrições PRIMARY KEY, UNIQUE, FOREIGN KEY e ENUM protegem a integridade dos dados; os índices aceleram as buscas mais comuns; as views simplificam consultas e limitam a exposição de colunas; as triggers garantem regras de status mesmo fora da aplicação; as procedures e functions concentram relatórios e protocolos no servidor; e os perfis de acesso fazem o banco refletir a hierarquia do sistema. Cada recurso foi adotado para resolver uma necessidade concreta, descrita nas seções anteriores.

Limitações e melhorias futuras, identificadas na análise do código e dos scripts:

- Senhas e credenciais. A coluna Usuario.Senha aceita 255 caracteres, espaço suficiente para um hash, mas hoje as senhas são comparadas em texto puro no login, e as credenciais dos usuários do banco estão fixas na classe DAO. O próximo passo é armazenar senhas com hash e salt e externalizar a configuração de conexão.
- Tabelas ainda não integradas à interface. Foto e Pergunta_Seguranca estão modeladas e criadas, mas ainda não são lidas nem gravadas pelas classes Java; da mesma forma, grupos_usuarios e rel_usuario_grupo existem, enquanto o perfil é hoje identificado pela presença do usuário nas tabelas Funcionário e Administrador. Integrá-las permitiria validar a posse por perguntas de segurança, anexar fotos aos objetos e unificar a autorização.
- Transações. Operações que envolvem mais de uma instrução (por exemplo, cadastrar o retirante, criar a reivindicação e registrar o log) executam em conexões separadas. Agrupá-las em uma única transação, como já é feito no cadastro de usuário, evitaria estados parciais.
- Normalização. A redundância de ID_Objeto e ID_Retirante em Devolucao poderia ser eliminada derivando esses dados de Reivindicacao, ou protegida por uma restrição ou trigger de consistência.
- Evolução. Também seriam úteis um índice de texto completo para a busca por descrição, uma rotina automática para marcar objetos com Data_Limite_Guarda vencida como encaminhados para doação e testes automatizados das camadas de controle.

Referências

CHEN, Peter Pin-Shan. The Entity-Relationship Model: Toward a Unified View of Data. ACM Transactions on Database Systems, v. 1, n. 1, p. 9–36, 1976.

DATE, C. J. Introdução a Sistemas de Bancos de Dados. 8. ed. Rio de Janeiro: Campus, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. Sistemas de Banco de Dados. 7. ed. São Paulo: Pearson, 2019.

ORACLE. MySQL 8.0 Reference Manual. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/ Acesso em: 01 de novembro de 2026

ORACLE. JDBC Basics. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/jdbc/basics/ 01 de novembro de 2026

ORACLE. Creating a GUI With Swing. The Java Tutorials. Disponível em: https://docs.oracle.com/javase/tutorial/uiswing/ 01 de novembro de 2026

SILBERSCHATZ, Abraham; KORTH, Henry F.; SUDARSHAN, S. Sistema de Banco de Dados. 6. ed. Rio de Janeiro: Elsevier, 2012.